

Cortex — Operating Intelligence for Startups

Turning a five-database Notion workspace into an agent that runs the founder's weekly operating cadence

Dev Shah · djshah2903@gmail.com · github.com/djs525/cortex-startup-oi (system prompt, commands, setup guides) · Built for the MLH Notion AI Challenge, 2026

The problem

Early-stage founders already keep their operating data in Notion — goals in one database, deals in another, investors and hiring in two more. The data is there. What is missing is anyone with time to read across it. The weekly review is the first thing to slip when the week gets busy, and it is precisely the thing that would have caught the deal that went quiet twelve days ago, or the lead investor who asked for a demo and never got it.

Every Notion "startup OS" I looked at answers this with more structure — more templates, more views, more dashboards — which improves the record-keeping and worsens the reading problem. The bottleneck is not capture. It is synthesis.

The bet

If the workspace already holds the facts, the review should not be a thing a founder writes. It should be a thing they read. So I inverted the artifact: rather than a template a human fills in, Cortex is an agent that reads the workspace and writes the review back into it. Two words — **run Cortex** — produce a dated weekly review page in Notion, health-scored and structured into eight fixed sections.

How it is built — three layers, zero application code

Zero lines of application code was a deliberate constraint, not a limitation — the reasoning is in design decisions below.

Layer 1	Notion workspace	Five linked databases — OKR Tracker, Sales Pipeline, Investor CRM, Hiring Pipeline, Weekly Review — wired with relations and rollups, surfaced in one Company Pulse dashboard. This is the schema.
Layer 2	System prompt	Plain-English instructions loaded into a Claude Project: the workspace map, the trigger commands, the health-scoring rules, and the writing conventions. The program, written in English rather than Python.
Layer 3	Claude via Notion MCP	Reads every database, cross-references across all of them, and writes pages back. This is the runtime.

The intended workflow

The core loop is a Monday-morning cadence. A founder opens Claude with no context loaded and types two words. What happens next is fully specified in the system prompt:

- **Read.** Cortex fetches every active database. Nothing is passed in by the user — no pasted context, no attached docs.
- **Score.** Company health is scored against explicit thresholds. **Green:** three or more OKRs on track, no deal silent 10+ days, no lead investor overdue. **Yellow:** one to two OKRs at risk, or a deal silent 7–14 days, or an investor overdue. **Red:** any OKR off track, or a deal in Negotiation silent 14+ days, or a lead investor silent 14+ days.
- **Cross-reference.** The part that carries the product. A stale deal is linked to the at-risk OKR it threatens; an investor ask is checked against whether it was ever delivered; a role open 35 days is tied to the revenue goal it blocks. No single database contains any of these facts.
- **Write.** A page titled Week of [date] is created in the Weekly Review database, marked Draft and flagged as generated. Eight sections, always in the same order: wins, blockers, OKR pulse, sales snapshot, hiring update, investor update, decisions made, next-week priorities. Sections are never skipped — an inactive module writes an explicit "not yet active" line instead.

- **Act.** Priorities come out as three to five numbered actions, each specific and assignable. The output is a decision list, not a status report.

Supporting commands

The weekly review is the anchor; the rest of the surface is deliberately narrow. Fifteen commands, split by whether they write to Notion or only answer in chat — an explicit distinction, so the founder always knows when the workspace is being modified.

Command	Writes ?	What it produces
run Cortex	Yes	The full weekly review, health-scored, eight sections.
stale deals	Yes	Deals silent past threshold, each with a follow-up drafted from that deal's own Notes context — not a generic template.
investor check	Yes	Overdue investors by tier, what each one asked for, whether it was delivered, and a drafted message.
loss analysis	Yes	Patterns across closed-lost deals; a loss blamed on a missing feature is tagged to re-engage when it ships.
activate [module]	No	Walks through adding Engineering, Product, People, Finance, or Marketing.

Design decisions worth defending

A system prompt instead of an application

The obvious build is a web app on the Notion API. I chose not to: the workspace already is the schema, and the hard part of this product is judgment — what counts as stale, what a follow-up should say, which two things matter this week. That is prompt work, and iterating on it takes seconds rather than a deploy cycle. The tradeoff is real — no auth, no multi-tenancy, no guaranteed determinism, setup falls to the user — but worth it to find out whether founders want this at all.

Hard-coded thresholds rather than model judgment

Staleness, urgency, and health scoring are fixed numeric rules in the prompt, not left to the model. If a founder is going to trust a red flag, it has to mean the same thing every week. The model's latitude is in the writing and cross-referencing; the alerting is deterministic.

Modules that stay off until they are earned

Nine modules exist; only the founder core is active on day one, so a three-person team gets five databases rather than fifteen. Engineering unlocks at the first engineering hire, People at roughly ten employees, Finance after a round — same commands throughout. Onboarding cost was the thing most likely to kill adoption, so it is what I optimized against.

Drafts, not sends

Every review lands as a Draft and every outreach email as a draft. The agent reads freely and writes into a review queue, but never sends on someone's behalf.

Honest status and what comes next

Cortex was built for a hackathon and is published as a duplicable Notion template with setup guides. It has not been run against a real company's live data over a sustained period, and I have no retention or usage numbers to show — treating a two-week build as a validated product would be the wrong read. Three things I would test first with real users:

- **Are the thresholds right?** Seven and ten days are guesses, not findings. Enterprise and SMB cycles likely need different numbers, which argues for per-workspace configuration.
- **Does the review get read, or archived?** The failure mode I expect is a well-written page nobody opens. The metric is not pages generated — it is whether next-week priorities show up as completed work.
- **Does setup survive a real workspace?** Forty-five minutes against a clean template is known; against a messy real Notion it is not, and that gap is where users drop.